

★ WEEK 10 · PART 2 of 3

The Three Problems and Their Algorithms

Evaluation, Decoding, and Learning

The forward algorithm · Viterbi · Baum-Welch · filtering vs smoothing

An HMM is only useful once you can compute with it. Three questions — how likely, which path, which parameters — are answered by three classic dynamic-programming algorithms over a trellis of states.

2

PART 2 · ALGORITHMS

Computing With Hidden States

Evaluation by the forward algorithm, decoding by Viterbi, learning by Baum-Welch, and the filtering-versus-smoothing distinction — now applied to discrete regimes.

We Have the Model — Now, the Computation

Part 1 built the HMM; Part 2 makes it work

1

The model is defined

Part 1 gave the pieces: an initial distribution, a transition matrix, and an emission per regime. But we never said how to actually compute anything.

2

Three questions arise

How likely is the data? Which regime path is most likely? And which parameters fit best? Everything you do with an HMM is one of these three.

3

Three algorithms answer them

The forward algorithm, Viterbi, and Baum-Welch — all efficient dynamic programs over a trellis of states across time.

Part 2 in one line: evaluate, decode, and learn — three problems, three trellis algorithms.

The Three Problems of an HMM

Everything you do is one of these

Every HMM task falls into one of three problems: evaluation (how likely is the data?), decoding (what was the hidden path?), and learning (what parameters fit?). Each has its own algorithm.

The three questions

- Evaluation: the likelihood of the observed data.
- Decoding: the most likely sequence of hidden regimes.
- Learning: the parameters that best explain the data.

Why frame it this way

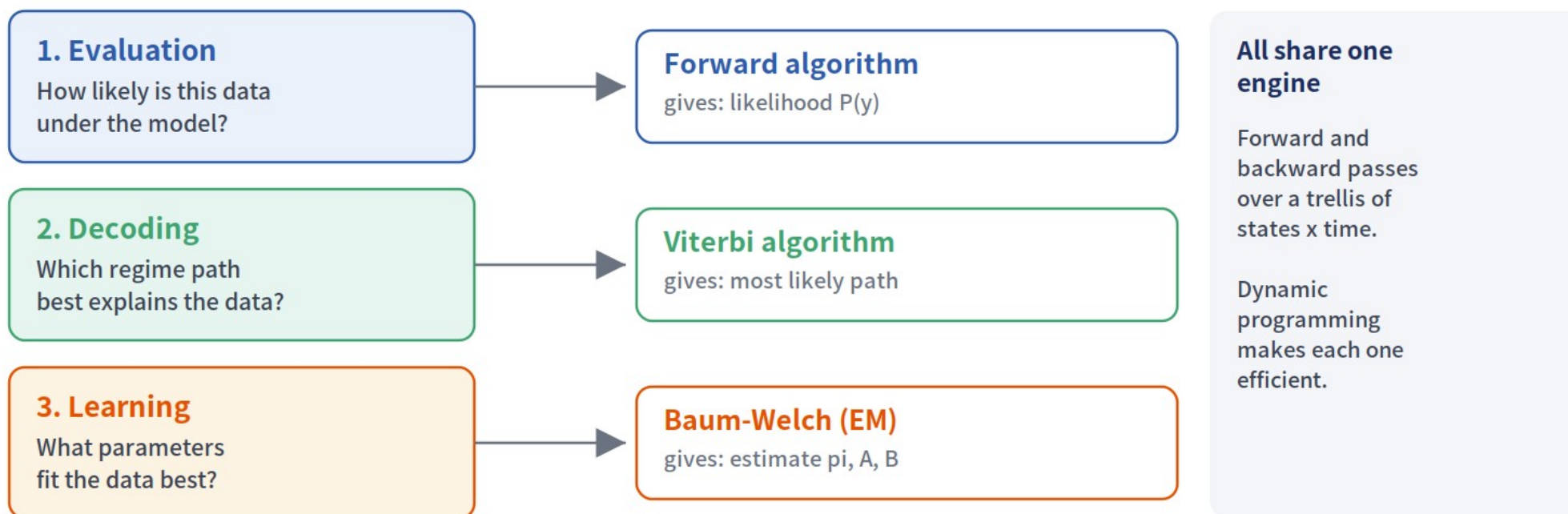
- It organizes everything HMMs can do.
- Each maps to one efficient algorithm.
- They build on each other — learning uses evaluation inside.

Three Questions, Three Algorithms

Evaluation is answered by the forward algorithm, decoding by Viterbi, and learning by Baum-Welch. All three are dynamic programs over the same trellis of states across time.

Three Questions, Three Algorithms

Everything you do with an HMM is one of these three problems.



Evaluation: How Likely Is This Data?

The likelihood under the model

The question

- Given a fitted HMM, how probable is an observed return series?
- This is the model's likelihood for that data.
- It is the basis for comparing and fitting models.

Why it matters

- It scores how well the model explains the data.
- It drives model selection — which HMM is better.
- Learning maximizes exactly this quantity.

Why the Naive Approach Fails

Enumerating every path is hopeless

The direct way to get the likelihood is to sum over every possible hidden path. But the number of paths grows exponentially with the length of the series — utterly infeasible for real data.

The blow-up

- With K regimes and T steps, there are K -to-the- T paths.
- Even a short series has astronomically many.
- Summing them one by one is impossible.

The way out

- Don't re-enumerate — reuse shared sub-computations.
- Dynamic programming collapses the exponential sum.
- That is exactly the forward algorithm.

The Forward Algorithm

Likelihood without enumerating every path

$$\alpha_t(j) = \left[\sum_i \alpha_{t-1}(i) A_{ij} \right] b_j(y_t) \quad \alpha_t(j) = P(y_{1:t}, X_t = j)$$

The forward variable

- Alpha at time t is the joint probability of the data so far AND being in state j .
- It is built recursively from the previous column.
- Each step: combine incoming paths, then apply the emission.

Why it is efficient

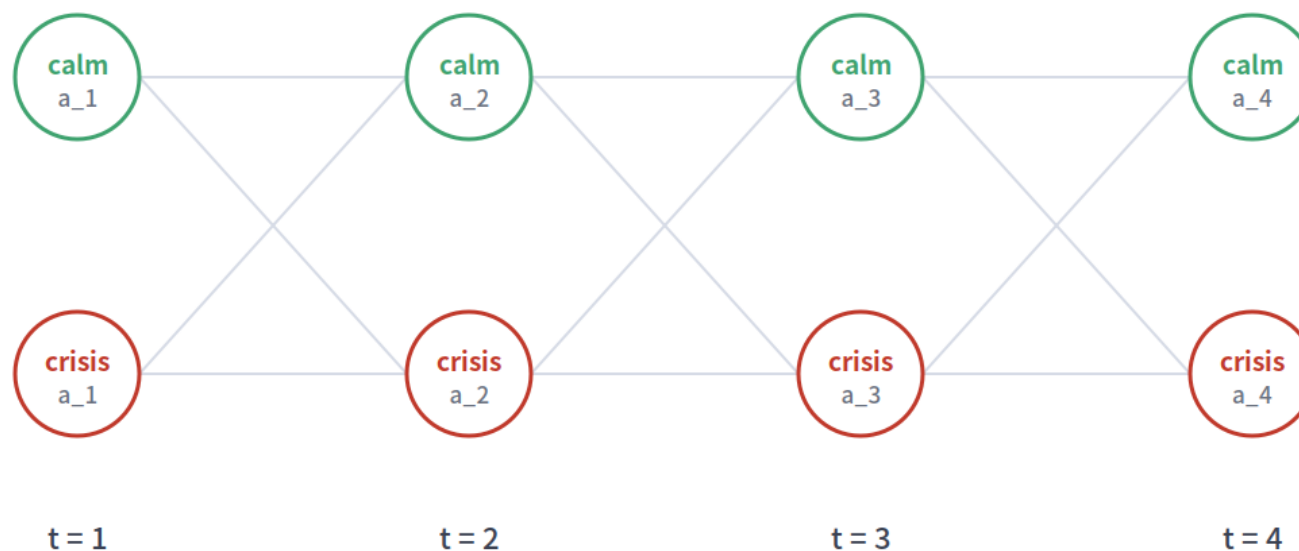
- Work grows linearly with the series length, not exponentially.
- Each column reuses the last — no re-enumeration.
- The whole exponential sum collapses to a simple recursion.

The Forward Trellis

States down, time across. Each node's value combines all incoming paths from the previous column times the transition, then the emission. The likelihood is the final column, summed.

The Forward Trellis

Sum over all paths, cheaply: each node reuses the last column instead of re-enumerating.



Each node's alpha = (sum of incoming alphas x transition) x emission. The likelihood is the last column, summed.

The Likelihood, Cheaply

Sum the final column

likelihood: $P(y_{1:T}) = \sum_i \alpha_T(i)$ (sum over paths, done cheaply)

The payoff

- Summing the forward variables at the last step gives the likelihood.
- The exponential path-sum is computed in linear time.
- One forward pass, and you have the model's score.

What it unlocks

- Comparing candidate HMMs by likelihood.
- The inner loop of Baum-Welch learning.
- The normalizer for regime probabilities.

Filtered Regime Probabilities

The probability of each regime, right now

$$\text{filtered regime prob: } P(X_t = i \mid y_{1:t}) = \frac{\alpha_t(i)}{\sum_j \alpha_t(j)}$$

What it is

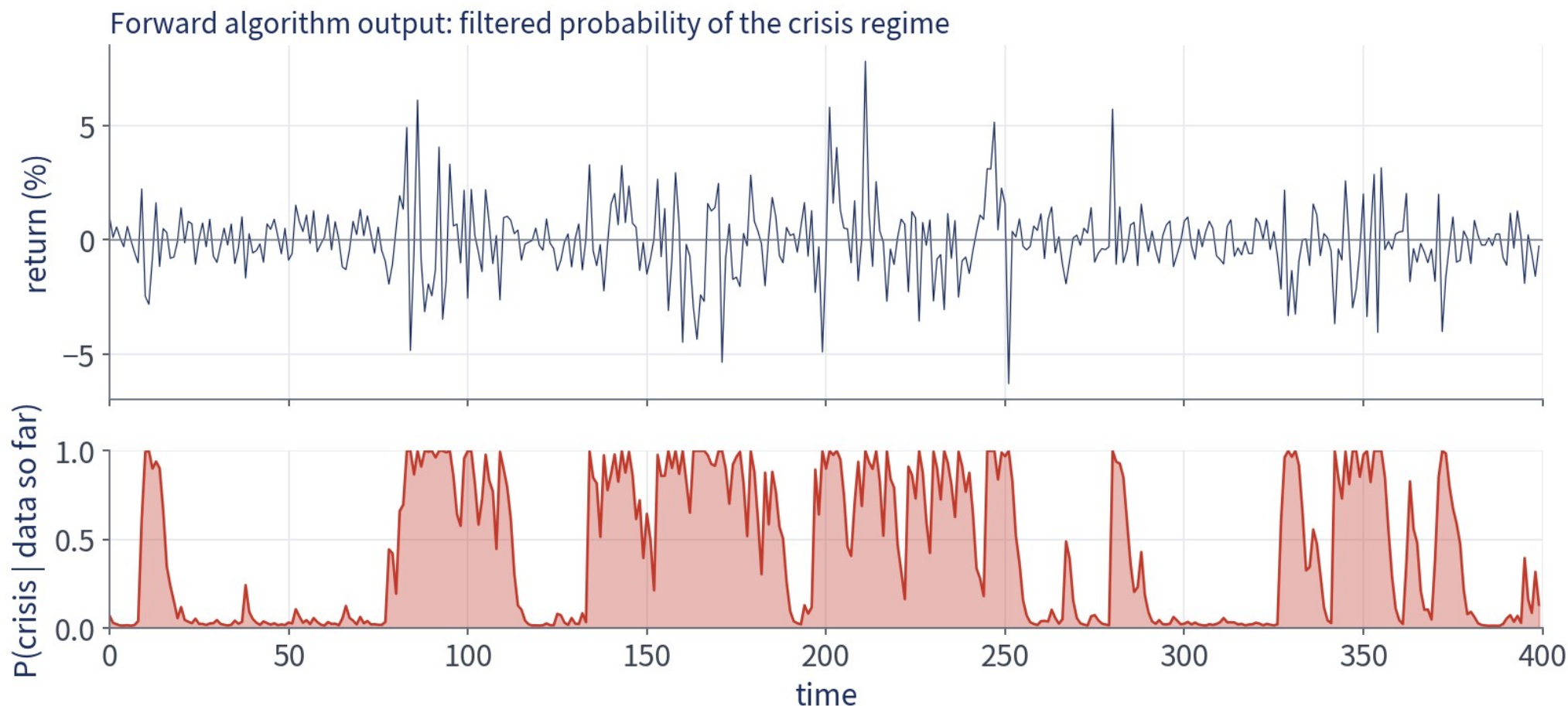
- Normalize the forward variables and you get a probability.
- It is the chance of each regime given data up to now.
- This is filtering — the Week 8 idea, for discrete states.

How you use it

- A running probability of being in the crisis regime.
- It updates causally as each return arrives.
- A live signal of the market's current mood.

Filtered Probability of Crisis

The forward algorithm gives, at each moment, the probability of being in the crisis regime given the data so far. It rises through turbulent stretches and falls back in calm ones.



Decoding: The Hidden Path

Which regime sequence explains the data?

The question

- Given the returns, what was the most likely sequence of regimes?
- Not just the probability each day — the best whole path.
- This labels history into calm and crisis stretches.

Why it matters

- It gives a single, coherent regime timeline.
- It is what you plot to 'see' the regimes.
- It underpins regime-aware decisions (Part 3).

Pointwise Best vs Single Best Path

Two different notions of 'most likely'

Pointwise (filtering/smoothing)

- Pick the most probable regime at EACH time separately.
- It can produce an inconsistent, jumpy path.
- Good for probabilities, not for a coherent sequence.

Single best path (Viterbi)

- Find the one sequence that is jointly most likely.
- It respects the transition structure throughout.
- This is the right tool for a decoded timeline.

The Viterbi Algorithm

The single most likely regime path

$$\text{Viterbi: } \delta_t(j) = \left[\max_i \delta_{t-1}(i) A_{ij} \right] b_j(y_t) \Rightarrow \text{backtrack the best path}$$

The idea

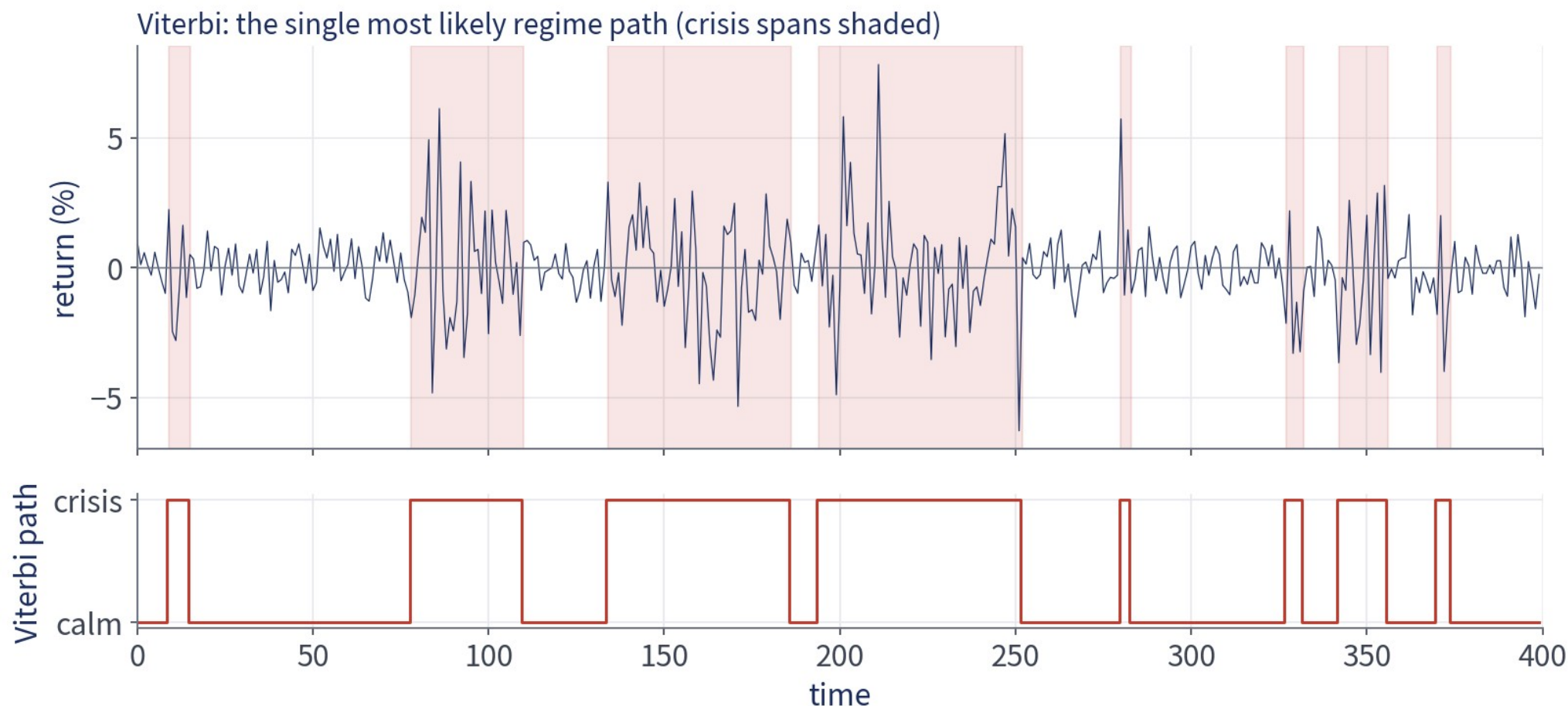
- Like the forward pass, but take a MAX instead of a sum.
- Track, at each node, the best incoming path.
- Then backtrack from the end to recover the sequence.

The result

- The globally most likely sequence of regimes.
- Coherent, transition-respecting, and unique.
- The standard way to decode an HMM.

The Viterbi Path

The single most likely regime sequence, decoded from the returns. Crisis spans (shaded) form coherent blocks — Viterbi respects the sticky transitions rather than flickering day to day.



Filtering vs Smoothing, for Regimes

The Week 8 distinction, now discrete

Filtering

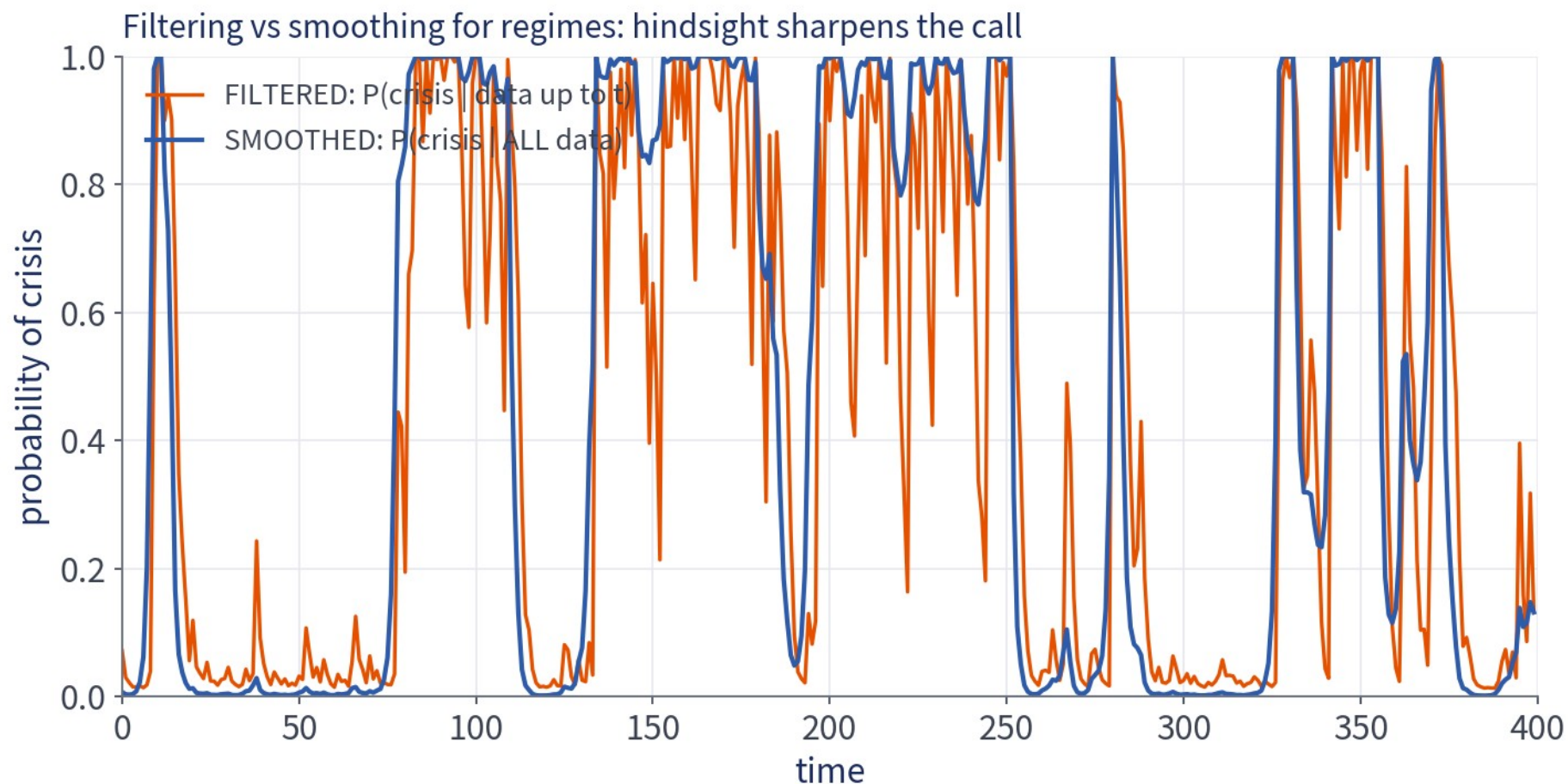
- The regime probability using data up to now.
- Causal and real-time — usable live.
- From the forward pass alone.

Smoothing

- The regime probability using the WHOLE sample.
- A backward pass is added to the forward one.
- Cleaner and more confident, but only in hindsight.

Filtered vs Smoothed Regimes

The filtered probability (orange) reacts in real time but wavers; the smoothed probability (blue), using all data in hindsight, is sharper and more decisive about when each regime held.



Learning: Fitting the Model

Estimating the parameters from data

So far we assumed the HMM's parameters were known. Learning is the problem of estimating them — the transition matrix and each regime's emission — from data in which the states themselves are never observed.

What we estimate

- The initial distribution over regimes.
- The transition matrix between regimes.
- Each regime's emission (its mean and volatility).

The difficulty

- The states are hidden, so we can't just count transitions.
- We must infer the states and the parameters together.
- This calls for an iterative approach.

The Chicken-and-Egg Problem

States need parameters; parameters need states

The circularity

- To estimate parameters, you'd want to know the regimes.
- To infer the regimes, you need the parameters.
- Neither is available at the start.

The resolution

- Guess parameters; infer regime probabilities; re-estimate; repeat.
- Each round improves the fit.
- This alternation is the EM algorithm.

Baum-Welch (EM)

Fitting when the states are hidden

Baum-Welch (EM): E-step: forward-backward $\rightarrow$ M-step: re-estimate (π, A, B)

The two steps

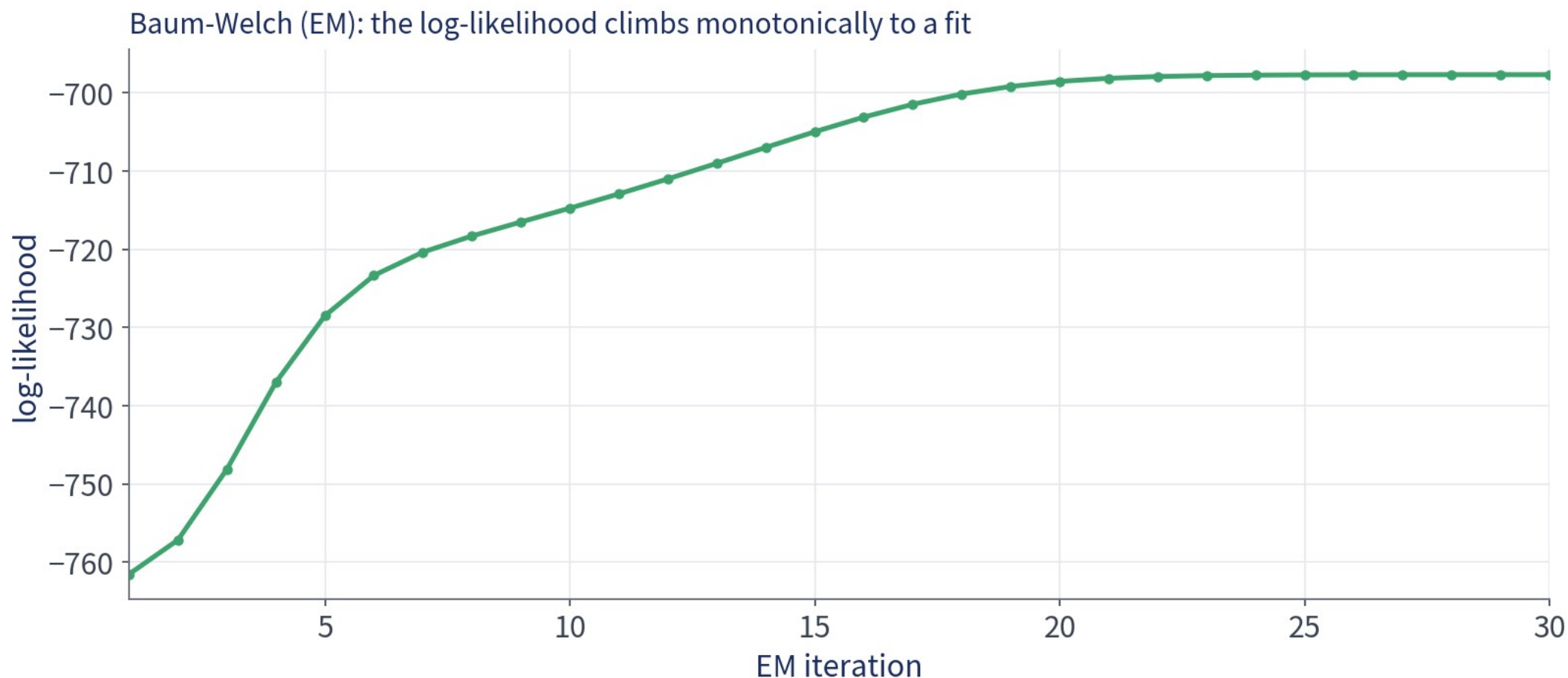
- E-step: run forward-backward to get regime probabilities.
- M-step: re-estimate the parameters using those probabilities.
- Alternate until the fit stops improving.

The guarantee

- Each iteration never decreases the likelihood.
- It climbs to a (local) maximum.
- It is the standard way to train an HMM.

EM Climbs the Likelihood

From a poor initial guess, each Baum-Welch iteration raises the log-likelihood, converging to a fitted model. The monotone climb is the algorithm's defining guarantee.



Forward, Viterbi, and Baum-Welch

Direct Claude Code through all three algorithms

GOAL Run the three HMM algorithms on a return series and see the likelihood, the decoded path, and the fit.

[INPUT] Prompt · Claude Code

Write and run `hmm_algorithms.py` that, on a 2-state Gaussian HMM fit to daily returns:

1. runs the FORWARD algorithm and reports the log-likelihood and the filtered $P(\text{crisis})$;
2. runs VITERBI and plots the decoded regime path;
3. compares FILTERED vs SMOOTHED regime probs;
4. runs BAUM-WELCH from a poor start and plots the log-likelihood climbing each iteration.

Confirm the likelihood never decreases across EM iterations, and that Viterbi gives coherent spans.

How to work with it

- Forward gives probabilities; Viterbi gives a path.
- Smoothed should be sharper than filtered.
- EM's log-likelihood must be monotone up.
- Watch for label switching between runs.

Deliverable: `hmm_algorithms.py` + likelihood, a decoded path, and an EM convergence plot.

Three Questions Before You Move On

If you can answer these, Part 2 has landed

Q1

What does the forward algorithm compute, and why not enumerate paths?

**Likelihood; paths
explode**

Q2

Viterbi differs from the forward pass by doing what?

Max instead of sum

Q3

What is guaranteed to happen to the likelihood in Baum-Welch?

It never decreases

All three come from the forward recursion, Viterbi's max, and EM's monotone climb.

Three Misconceptions to Avoid

Where HMM algorithms trip people up

Common myths

- "Stringing together the per-day best regimes gives Viterbi."
- "Baum-Welch finds the global best parameters."
- "Filtering and smoothing give the same regime call."

The reality

- That pointwise path can be incoherent; Viterbi optimizes jointly.
- EM only reaches a LOCAL maximum — initialization matters.
- Smoothing uses future data, so it is sharper than filtering.

Three Things to Carry Forward

Part 2 in one screen

1

Evaluation — the forward algorithm

It computes the data's likelihood without enumerating exponentially many paths, and its normalized form gives the filtered probability of each regime in real time.

2

Decoding — Viterbi

It finds the single most likely regime path by taking a max instead of a sum and backtracking. That coherent path is how we read regimes off the returns.

3

Learning — Baum-Welch (EM)

Alternating a forward-backward E-step with a re-estimation M-step fits the parameters when the states are hidden, climbing the likelihood to a local maximum.

Next — Part 3: how many regimes, regime-aware pairs trading (Stage IV), and the cautions.